

Réseau de neurones pour le Fanorona 3×3

Fondements mathématiques et implémentation avancée

Une approche généralisée de l'apprentissage automatique appliqué aux jeux
traditionnels

MIT - L3

RAMAROSON Tojotiana Cyriaque

RANDRIANARISOA Tsantamirindra

RAKOTOARIJAONA Aro Mino Avotra

RANAIVO Ny Aina Peniala

Résumé

Ce document présente une analyse de l'architecture, de l'implémentation et des fondements mathématiques d'un réseau de neurones artificiels développé spécifiquement pour le jeu traditionnel malgache *Fanorona* sur plateau 3×3 . L'étude couvre les aspects théoriques profonds incluant l'encodage vectoriel des états de jeu, la formulation des algorithmes de propagation avant et arrière, les justifications mathématiques des choix d'activation (ReLU, softmax), d'initialisation (Xavier), et de fonction de perte (entropie croisée). Le rapport détaille également la génération synthétique des données d'entraînement et présente l'implémentation modulaire en Rust avec traitement par mini-batch. L'architecture générique permet une configuration flexible des couches, utilisant la fonction ReLU pour les couches cachées et softmax pour la couche de sortie, produisant 81 logits correspondant aux 81 coups possibles (9×9 positions départ-arrivée).

Table des matières

1	Introduction et contexte	1
1.1	Objectifs et motivation	1
1.2	Architecture générique et spécialisations	1
2	Modélisation mathématique du jeu	2
2.1	Formalisation de l'espace d'états	2
2.1.1	Représentation discrète du plateau	2
2.1.2	Formalisation de l'objectif d'apprentissage	3
2.2	Encodage vectoriel et plongement	3
2.2.1	Transformation one-hot	3
2.2.2	Construction du vecteur d'entrée	3
2.3	Génération synthétique des données	4
3	Architecture du réseau neuronal	5
3.1	Formalisme mathématique général	5
3.1.1	Notations et conventions	5
3.1.2	Illustration architecturale améliorée	5
3.1.3	Propagation avant : formulation mathématique	6
3.2	Fonctions d'activation et leurs propriétés	6
3.2.1	Fonction ReLU pour les couches cachées	6
3.2.2	Softmax pour la couche de sortie	7
4	Fonction de perte et optimisation	8

4.1	Construction de l'étiquette cible	8
4.2	Fonction de perte : entropie croisée	8
4.3	Algorithme de rétropropagation	8
4.3.1	Gradient en sortie : propriété fondamentale	9
4.3.2	Rétropropagation dans les couches cachées	10
4.3.3	Gradients des paramètres	11
5	Initialisation et stabilité numérique	12
5.1	Initialisation Xavier : fondements théoriques	12
5.1.1	Justification mathématique	12
5.2	Algorithme d'optimisation par mini-batch	12
5.2.1	Descente de gradient stochastique par mini-batch	13
5.2.2	Implémentation matricielle par batch	13
5.2.3	Avantages du traitement par batch	13
6	Sélection de coups et évaluation	14
6.1	Stratégie de décision	14
7	Analyse de performance et validation	15
7.1	Métriques d'évaluation	15
7.1.1	Précision globale	15
7.1.2	Taux de coups valides	15
7.2	Fonction de perte au cours de l'entraînement	15
7.3	Validation croisée et overfitting	16
8	Implémentation et aspects techniques	17
8.1	Architecture modulaire en Rust	17
8.2	Bibliothèques utilisées	17

8.3	Optimisations numériques	17
8.3.1	Stabilité du softmax	18
8.3.2	Gestion de la précision	18
8.4	Configuration et hyperparamètres	18
8.5	Sauvegarde et chargement des modèles	18
9	Interface utilisateur et mode jeu	20
9.1	Commandes principales	20
9.1.1	Mode entraînement	20
9.1.2	Mode jeu	20
9.2	Format des coups	20
A	Dérivations mathématiques complètes	22
A.1	Récapitulatif des notations	22
A.2	Dérivées explicites pour la couche de sortie ($k = L$)	22
A.3	Dérivées pour les couches cachées ($k < L$)	23
A.4	Formulation matricielle par batch	23
B	Extensions et perspectives	24
B.1	Améliorations possibles	24
B.1.1	Techniques d'optimisation avancées	24
B.1.2	Architectures alternatives	24
B.1.3	Amélioration de la représentation	24
B.2	Applications à d'autres jeux	24
B.3	Apprentissage par renforcement	25
C	Conclusion	26

Chapitre 1

Introduction et contexte

1.1 Objectifs et motivation

Le développement de systèmes d'intelligence artificielle capables de maîtriser des jeux complexes constitue un domaine de recherche fondamental en apprentissage automatique. Ce projet vise à concevoir, analyser théoriquement et implémenter un réseau de neurones capable d'imiter des stratégies optimales pour le jeu Fanorona sur un plateau réduit de dimensions 3×3 .

L'approche adoptée privilégie une double perspective : d'une part, l'exposition rigoureuse des fondements mathématiques sous-jacents au modèle et aux algorithmes d'apprentissage ; d'autre part, la traduction pratique de ces formulations théoriques dans une implémentation performante en Rust avec optimisations batch.

1.2 Architecture générique et spécialisations

Le modèle développé présente une architecture générique paramétrable par une liste de tailles de couches, offrant ainsi une flexibilité maximale pour l'expérimentation. Dans l'usage pratique du projet :

- **Couches cachées** : activation par fonction ReLU $\text{ReLU}(z) = \max(0, z)$
- **Couche de sortie** : production de 81 logits suivie de l'application d'un softmax global
- **Représentation unifiée** : un seul vecteur de probabilités pour tous les coups possibles (d, a)

Cette architecture unifiée permet une modélisation naturelle de l'espace des coups où chaque coup (d, a) est indexé par $9d + a$, produisant un espace de sortie de dimension 81.

Chapitre 2

Modélisation mathématique du jeu

2.1 Formalisation de l'espace d'états

2.1.1 Représentation discrète du plateau

Soit $\mathcal{P} = \{0, 1, 2, \dots, 8\}$ l'ensemble des positions sur le plateau 3×3 . Chaque case $i \in \mathcal{P}$ est caractérisée par un état s_i appartenant à l'ensemble discret :

L'espace d'états par case est défini comme :

$$\mathcal{S} = \{-2, -1, 0, 1, 2\}$$

où :

$$s_i = -2 \iff \text{pion du joueur A déjà déplacé en position } i \quad (2.1.1)$$

$$s_i = -1 \iff \text{pion du joueur A (non déplacé) en position } i \quad (2.1.2)$$

$$s_i = 0 \iff \text{case } i \text{ vide} \quad (2.1.3)$$

$$s_i = 1 \iff \text{pion du joueur B en position } i \quad (2.1.4)$$

$$s_i = 2 \iff \text{pion du joueur B déjà déplacé en position } i \quad (2.1.5)$$

L'état global du plateau est représenté par le vecteur $\mathbf{s} = (s_0, s_1, \dots, s_8, p) \in \mathcal{S}^9 \times \{1, 2\}$ où p désigne le joueur courant.

2.1.2 Formalisation de l'objectif d'apprentissage

Problème de prédiction optimale

Soit $\mathcal{X} = \mathcal{S}^9 \times \{1, 2\}$ l'espace des configurations (état du plateau $\times$ joueur courant).
L'objectif du réseau de neurones est d'apprendre la fonction :

$$f^* : \mathcal{X} \rightarrow \mathcal{P} \times \mathcal{P}$$

qui associe à chaque configuration le coup optimal (d^*, a^*) selon une politique experte (ex : Minimax).

2.2 Encodage vectoriel et plongement

2.2.1 Transformation one-hot

L'encodage one-hot constitue une injection de l'espace discret $\mathcal{S}$ vers un sous-espace de $\mathbb{R}^5$:

Fonction d'encodage one-hot

$$\phi : \mathcal{S} \rightarrow \{0, 1\}^5$$

définie par :

$$\phi(s) = \begin{cases} [0, 0, 0, 1, 0]^T & \text{si } s = -2 \\ [0, 0, 0, 0, 1]^T & \text{si } s = -1 \\ [1, 0, 0, 0, 0]^T & \text{si } s = 0 \\ [0, 1, 0, 0, 0]^T & \text{si } s = 1 \\ [0, 0, 1, 0, 0]^T & \text{si } s = 2 \end{cases}$$

Propriétés de l'encodage one-hot

L'encodage one-hot ϕ satisfait les propriétés suivantes :

1. **Injectivité** : ϕ est une injection, i.e., $\phi(s_1) = \phi(s_2) \Rightarrow s_1 = s_2$
2. **Orthogonalité** : $\forall s_1 \neq s_2, \langle \phi(s_1), \phi(s_2) \rangle = 0$
3. **Normalisation** : $\forall s \in \mathcal{S}, \|\phi(s)\|_2 = 1$

2.2.2 Construction du vecteur d'entrée

Le vecteur d'entrée du réseau est construit par concaténation :

$$\mathbf{x} = \begin{bmatrix} \phi(s_0) \\ \phi(s_1) \\ \vdots \\ \phi(s_8) \\ p-1 \end{bmatrix} \in \mathbb{R}^{46}$$

où $p \in \{1, 2\}$ identifie le joueur courant, et $(p-1) \in \{0, 1\}$ est ajouté comme dernière composante.

Cette représentation évite l'introduction d'un ordre numérique arbitraire entre les différents états de cases, préservant ainsi la structure catégorielle naturelle du problème.

2.3 Génération synthétique des données

Étant donné l'absence de datasets publics pour le Fanorona 3×3, l'ensemble d'entraînement a été synthétisé algorithmiquement. Les étiquettes (d^*, a^*) sont générées par un oracle externe (algorithme Minimax, heuristique experte) appliqué aux configurations de jeu.

Cette approche de génération automatique présente l'avantage de produire un dataset équilibré et exhaustif, couvrant une large gamme de configurations possibles.

Chapitre 3

Architecture du réseau neuronal

3.1 Formalisme mathématique général

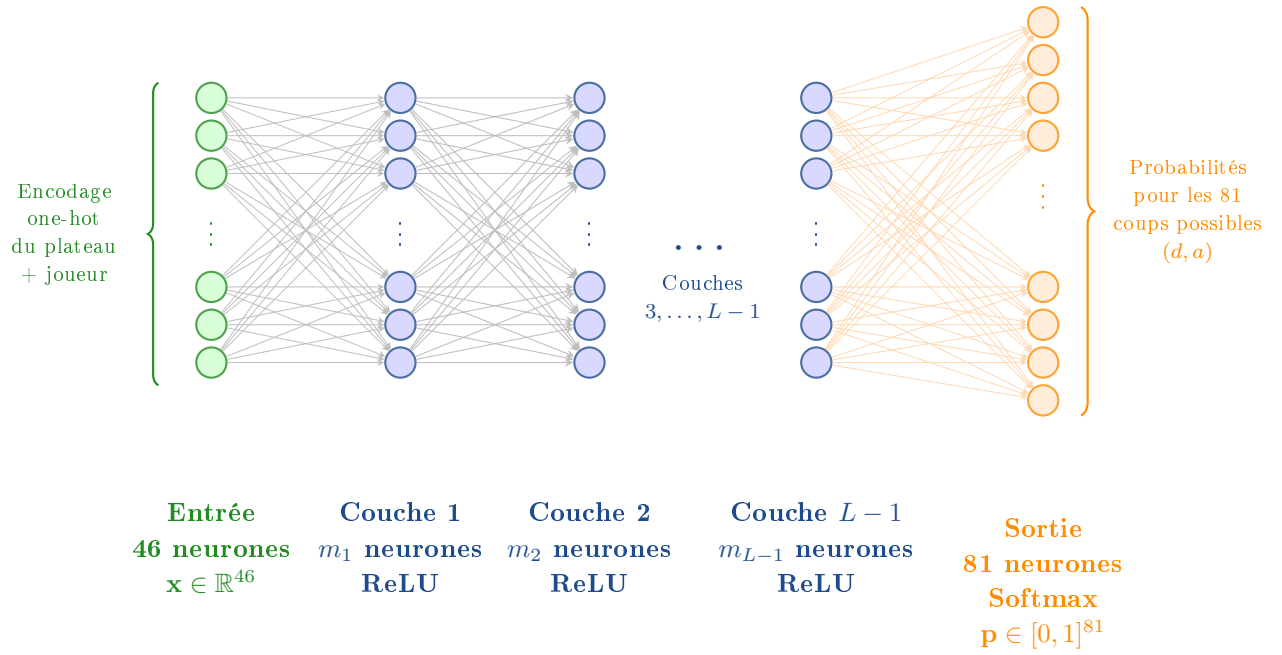
3.1.1 Notations et conventions

Soit $L \in \mathbb{N}^*$ le nombre total de couches (incluant la couche de sortie). Pour chaque couche $k \in \{1, 2, \dots, L\}$:

- $W^{(k)} \in \mathbb{R}^{m_k \times n_k}$: matrice des poids de connexion
- $\mathbf{b}^{(k)} \in \mathbb{R}^{m_k}$: vecteur de biais
- $\mathbf{z}^{(k)} \in \mathbb{R}^{m_k}$: entrée nette (avant activation)
- $\mathbf{a}^{(k)} \in \mathbb{R}^{m_k}$: sortie activée
- m_k : nombre de neurones dans la couche k
- n_k : dimension de l'entrée de la couche k

avec la convention $\mathbf{a}^{(0)} = \mathbf{x}$ (vecteur d'entrée du réseau).

3.1.2 Illustration architecturale améliorée



3.1.3 Propagation avant : formulation mathématique

Algorithme de propagation avant

Pour chaque couche $k \in \{1, 2, \dots, L\}$, le calcul de propagation avant s'effectue selon :

$$\mathbf{z}^{(k)} = \mathbf{W}^{(k)} \mathbf{a}^{(k-1)} + \mathbf{b}^{(k)} \quad (3.1.1)$$

$$\mathbf{a}^{(k)} = \phi^{(k)}(\mathbf{z}^{(k)}) \quad (3.1.2)$$

où $\phi^{(k)}$ désigne la fonction d'activation de la couche k .

3.2 Fonctions d'activation et leurs propriétés

3.2.1 Fonction ReLU pour les couches cachées

Fonction ReLU

La fonction Rectified Linear Unit (ReLU) est définie par :

$$\text{ReLU}(z) = \max(0, z)$$

avec pour dérivée :

$$\text{ReLU}'(z) = \begin{cases} 1 & \text{si } z > 0 \\ 0 & \text{si } z \leq 0 \end{cases}$$

Propriétés de ReLU

La fonction ReLU satisfait :

1. **Non-linéarité** : ReLU introduit une non-linéarité tout en préservant la linéarité pour $z > 0$
2. **Sparse activation** : Pour toute entrée, seule une partie des neurones est activée
3. **Gradient non-saturant** : Pas de problème de gradient vanishing pour $z > 0$
4. **Efficacité computationnelle** : Calcul très rapide (simple comparaison)

3.2.2 Softmax pour la couche de sortie

Pour la couche de sortie ($k = L$), les logits $\mathbf{z}^{(L)} \in \mathbb{R}^{81}$ sont transformés par la fonction softmax :

Fonction softmax

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=0}^{80} e^{z_j}}, \quad i \in \{0, 1, \dots, 80\}$$

La sortie finale est : $\mathbf{a}^{(L)} = \text{softmax}(\mathbf{z}^{(L)}) \in [0, 1]^{81}$ avec $\sum_{i=0}^{80} a_i^{(L)} = 1$.

Chapitre 4

Fonction de perte et optimisation

4.1 Construction de l'étiquette cible

Vecteur d'étiquetage

Pour un exemple d'entraînement avec étiquette $(d^*, a^*) \in \mathcal{P} \times \mathcal{P}$, le vecteur cible $\mathbf{y} \in \{0, 1\}^{81}$ est défini par :

$$y_i = \begin{cases} 1 & \text{si } i = 9d^* + a^* \\ 0 & \text{sinon} \end{cases}$$

Cette construction encode le coup optimal (d^*, a^*) comme l'indice unique $9d^* + a^*$ dans un vecteur one-hot de dimension 81.

4.2 Fonction de perte : entropie croisée

Fonction de perte entropie croisée

La fonction de perte utilisée est l'entropie croisée catégorielle :

$$\mathcal{L}(\mathbf{x}; d^*, a^*) = - \sum_{i=0}^{80} y_i \log a_i^{(L)} = - \log a_{9d^*+a^*}^{(L)}$$

Cette formulation présente plusieurs avantages théoriques :

- **Convexité** par rapport aux paramètres de la dernière couche
- **Gradient non-nul** même pour des prédictions très confiantes mais erronées
- **Interprétation probabiliste** naturelle

4.3 Algorithme de rétropropagation

4.3.1 Gradient en sortie : propriété fondamentale

Gradient simplifié softmax avec entropie croisée

Pour la combinaison softmax avec entropie croisée, le gradient par rapport aux logits se simplifie remarquablement :

$$\delta_i^{(L)} \equiv \frac{\partial \mathcal{L}}{\partial z_i^{(L)}} = a_i^{(L)} - y_i, \quad \forall i \in \{0, 1, \dots, 80\}$$

Démonstration. La démonstration suit le même raisonnement que précédemment. Sachant que la fonction softmax est définie par :

$$a_i = \frac{e^{z_i}}{\sum_{j=0}^{80} e^{z_j}}$$

et que la fonction de perte est :

$$\mathcal{L} = -\log(a_{9d^*+a^*})$$

On calcule :

$$\frac{\partial \mathcal{L}}{\partial z_i} = \frac{\partial}{\partial z_i} \left(-\log(a_{9d^*+a^*}) \right)$$

Cas 1 : $i = 9d^* + a^*$

$$\frac{\partial}{\partial z_i} \left(-\log(a_i) \right) = -\frac{1}{a_i} \cdot a_i(1 - a_i) = a_i - 1$$

Cas 2 : $i \neq 9d^* + a^*$

$$\frac{\partial}{\partial z_i} \left(-\log(a_{9d^*+a^*}) \right) = -\frac{1}{a_{9d^*+a^*}} \cdot (-a_{9d^*+a^*} a_i) = a_i$$

D'où,

$$\delta_i^{(L)} = a_i^{(L)} - y_i, \quad \forall i \in \{0, 1, \dots, 80\}$$

□

4.3.2 Rétropropagation dans les couches cachées

Formule de rétropropagation pour ReLU

Pour une couche cachée $k < L$ avec activation ReLU :

$$\delta_j^{(k-1)} = \left(\sum_{i=1}^{n_k} \delta_i^{(k)} W_{ij}^{(k)} \right) \cdot \mathbb{K}_{z_j^{(k-1)} > 0}$$

où $\mathbb{K}_{z>0}$ est la fonction indicatrice.

Démonstration. Soit une couche k du réseau. Pour tout neurone i de cette couche on pose :

$$z_i^{(k)} = \sum_{j=1}^{n_{k-1}} W_{ij}^{(k)} a_j^{(k-1)} + b_i^{(k)}, \quad a_i^{(k)} = \text{ReLU}(z_i^{(k)}) = \max(0, z_i^{(k)})$$

Nous voulons obtenir $\delta_j^{(k-1)} := \frac{\partial \mathcal{L}}{\partial z_j^{(k-1)}}$ en fonction des quantités connues en couche k .

Dérivée de la perte par rapport à l'activation $a_j^{(k-1)}$. Par la règle de la chaîne, $a_j^{(k-1)}$ n'apparaît que dans les pré-activations $z_i^{(k)}$ de la couche suivante, donc $\frac{\partial \mathcal{L}}{\partial a_j^{(k-1)}} = \sum_{i=1}^{n_k} \frac{\partial \mathcal{L}}{\partial z_i^{(k)}} \frac{\partial z_i^{(k)}}{\partial a_j^{(k-1)}}$

Or $z_i^{(k)} = \sum_t W_{it}^{(k)} a_t^{(k-1)} + b_i^{(k)}$, donc $\frac{\partial z_i^{(k)}}{\partial a_j^{(k-1)}} = W_{ij}^{(k)}$

D'où : $\frac{\partial \mathcal{L}}{\partial a_j^{(k-1)}} = \sum_{i=1}^{n_k} \delta_i^{(k)} W_{ij}^{(k)}$

Dérivée de ReLU. La dérivée de ReLU est : $\text{ReLU}'(z) = \begin{cases} 1 & \text{si } z > 0 \\ 0 & \text{sinon} \end{cases} = \mathbb{K}_{z>0}$

Passage de $\partial \mathcal{L} / \partial a_j^{(k-1)}$ à $\partial \mathcal{L} / \partial z_j^{(k-1)}$. On applique la chaîne sur $a_j^{(k-1)} = \text{ReLU}(z_j^{(k-1)})$:

$$\begin{aligned} \delta_j^{(k-1)} &= \frac{\partial \mathcal{L}}{\partial z_j^{(k-1)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(k-1)}} \cdot \frac{da_j^{(k-1)}}{dz_j^{(k-1)}} \\ &= \left(\sum_{i=1}^{n_k} \delta_i^{(k)} W_{ij}^{(k)} \right) \cdot \mathbb{K}_{z_j^{(k-1)} > 0} \end{aligned}$$

D'où la formule : $\delta_j^{(k-1)} = \left(\sum_{i=1}^{n_k} \delta_i^{(k)} W_{ij}^{(k)} \right) \cdot \mathbb{K}_{z_j^{(k-1)} > 0}$ □

4.3.3 Gradients des paramètres

Gradients des poids et biais

Les gradients des paramètres de chaque couche k sont donnés par :

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(k)}} = \delta_i^{(k)} \cdot a_j^{(k-1)} \quad (4.3.1)$$

$$\frac{\partial \mathcal{L}}{\partial b_i^{(k)}} = \delta_i^{(k)} \quad (4.3.2)$$

Chapitre 5

Initialisation et stabilité numérique

5.1 Initialisation Xavier : fondements théoriques

Initialisation Xavier uniforme

Pour une couche avec n_{in} entrées et n_{out} sorties, l'initialisation Xavier uniforme tire les poids selon : $W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}\right)$

Cette initialisation vise à maintenir la variance des activations approximativement constante à travers les couches, évitant ainsi les problèmes de gradient qui disparaît ou explose lors de l'entraînement de réseaux profonds.

5.1.1 Justification mathématique

L'objectif de l'initialisation Xavier est de satisfaire : $\text{Var}(a_i^{(k)}) \approx \text{Var}(a_j^{(k-1)})$, $\forall k$

Pour une activation linéaire $z = \sum_j W_{ij} a_j$, sous l'hypothèse d'indépendance : $\text{Var}(z) = \sum_j W_{ij}^2 \text{Var}(a_j) \approx n_{\text{in}} \cdot \text{Var}(W) \cdot \text{Var}(a)$

Pour maintenir $\text{Var}(z) = \text{Var}(a)$, il faut : $n_{\text{in}} \cdot \text{Var}(W) = 1 \Rightarrow \text{Var}(W) = \frac{1}{n_{\text{in}}}$

L'initialisation Xavier généralise cette condition en considérant également la propagation arrière, d'où la moyenne harmonique $\frac{2}{n_{\text{in}}+n_{\text{out}}}$.

5.2 Algorithme d'optimisation par mini-batch

5.2.1 Descente de gradient stochastique par mini-batch

Le modèle d'apprentissage utilise des *mini-batches* de positions du jeu au lieu d'une seule position à chaque mise à jour. Chaque mini-batch contient un sous-ensemble aléatoire de positions extraites du jeu d'entraînement : $B_t = \{(x_i, y_i)\}_{i=1}^m \subset D$ où m est la taille du batch, et D l'ensemble total des positions d'entraînement.

Gradient de batch

Pour chaque batch B_t , les gradients individuels $\nabla_{\theta} \mathcal{L}(x_i, y_i; \theta)$ sont d'abord calculés et sommés, puis moyennés avant d'être appliqués aux poids du réseau : $\nabla_{\theta} \mathcal{L}_{B_t}(\theta) = \frac{1}{m} \sum_{(x_i, y_i) \in B_t} \nabla_{\theta} \mathcal{L}(x_i, y_i; \theta)$ La mise à jour des paramètres devient : $\theta_{t+1} = \theta_t - \alpha_t \nabla_{\theta} \mathcal{L}_{B_t}(\theta_t)$ où α_t est le taux d'apprentissage.

5.2.2 Implémentation matricielle par batch

L'implémentation utilise des opérations matricielles pour traiter simultanément tous les exemples d'un batch :

- $X \in \mathbb{R}^{m \times n_0}$: matrice des entrées (batch)
- $Y \in \mathbb{R}^{m \times 81}$: matrice des étiquettes (batch)
- $Z^{(k)} \in \mathbb{R}^{m \times n_k}$: pré-activations du batch
- $A^{(k)} \in \mathbb{R}^{m \times n_k}$: activations du batch

La propagation avant par batch s'écrit : $Z^{(k)} = A^{(k-1)}W^{(k)} + \mathbf{1}_m \otimes \mathbf{b}^{(k)T}$ $A^{(k)} = \phi^{(k)}(Z^{(k)})$ où $\mathbf{1}_m$ est le vecteur colonne de m uns et $\otimes$ le produit extérieur.

5.2.3 Avantages du traitement par batch

1. **Parallélisation** : exploitation des bibliothèques BLAS optimisées
2. **Stabilité** : réduction de la variance du gradient
3. **Généralisation** : régularisation implicite
4. **Convergence** : trajectoire d'optimisation plus stable

Chapitre 6

Sélection de coups et évaluation

6.1 Stratégie de décision

Prédiction du meilleur coup

Le réseau produit un vecteur de probabilités $\mathbf{p} = \mathbf{a}^{(L)} \in [0, 1]^{81}$ où chaque composante p_i représente la probabilité que le coup indexé par i soit optimal.

Le coup prédit $(d_{\text{pred}}, a_{\text{pred}})$ est obtenu par : $i^* = \arg \max_{i \in \{0, \dots, 80\}} p_i$ $d_{\text{pred}} = \lfloor i^*/9 \rfloor$, $a_{\text{pred}} = i^* \bmod 9$

Chapitre 7

Analyse de performance et validation

7.1 Métriques d'évaluation

7.1.1 Précision globale

$$\text{Précision}_{\text{coup}} = \frac{\text{Nombre de coups } (d,a) \text{ correctement prédits}}{\text{Nombre total d'exemples}}$$

7.1.2 Taux de coups valides

$$\text{Validité} = \frac{\text{Nombre de coups prédits légaux}}{\text{Nombre total de prédictions}}$$

Cette métrique mesure la capacité du réseau à apprendre les contraintes du jeu.

7.2 Fonction de perte au cours de l'entraînement

La fonction de perte moyenne par époque est calculée comme : $\mathcal{L}_{\text{epoch}} = \frac{1}{|\mathcal{B}|} \sum_{B \in \mathcal{B}} \mathcal{L}_B$ où $\mathcal{B}$ est l'ensemble des batches de l'époque.

Une décroissance monotone de $\mathcal{L}_{\text{epoch}}$ indique une convergence saine. Des oscillations importantes peuvent signaler un taux d'apprentissage trop élevé.

7.3 Validation croisée et overfitting

Étant donné la nature synthétique des données, une attention particulière doit être portée à :

- La diversité des configurations générées
- L'équilibre entre les différentes phases de jeu
- La validation sur des positions non vues pendant l'entraînement
- Le suivi de la perte sur un ensemble de validation séparé

Chapitre 8

Implémentation et aspects techniques

8.1 Architecture modulaire en Rust

L'implémentation est structurée en modules spécialisés :

- `neural.rs` : Noyau du réseau neuronal et opérations matricielles
 - Structure `Neural` contenant les paramètres
 - Méthodes `batch_forward`, `batch_grads`, `batch_backward`
 - Initialisation Xavier avec `xavier()`
 - Sérialisation/désérialisation binaire
- `fanorona3.rs` : Logique du jeu Fanorona
 - Structure `Fanorontelo` représentant l'état du jeu
 - Fonction `one_hot_fanorona()` pour l'encodage
 - Validation des coups avec `valid_fn3_move()`
 - Génération des coups possibles
- `dataset.rs` : Chargement et préparation des données
 - Fonction `load_dataset()` pour charger depuis fichier
 - Parsing des données d'entraînement
- `main.rs` : Point d'entrée et orchestration
 - Configuration des hyperparamètres
 - Boucle d'entraînement principale
 - Interface de jeu contre le modèle

8.2 Bibliothèques utilisées

8.3 Optimisations numériques

Bibliothèque	Utilisation
<code>ndarray</code>	Opérations matricielles et tensorielles
<code>ndarray-rand</code>	Initialisation aléatoire des poids
<code>serde</code>	Sérialisation/désérialisation des paramètres
<code>bincode</code>	Format binaire compact pour la sauvegarde

TABLE 8.1 – Dépendances principales du projet

8.3.1 Stabilité du softmax

Pour éviter les débordements numériques, l'implémentation utilise la forme stabilisée :

$$\text{softmax}(z_i) = \frac{e^{z_i - z_{\max}}}{\sum_j e^{z_j - z_{\max}}} \text{ où } z_{\max} = \max_j z_j.$$

Cette transformation ne change pas le résultat car : $\frac{e^{z_i - c}}{\sum_j e^{z_j - c}} = \frac{e^{-c} e^{z_i}}{e^{-c} \sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}}$

8.3.2 Gestion de la précision

- Utilisation de types flottants double précision (`f64`)
- Clipping des logarithmes : $\log(x) \rightarrow \log(\max(x, 10^{-12}))$ pour éviter $\log(0)$
- Normalisation des entrées lors du chargement des données

8.4 Configuration et hyperparamètres

Hyperparamètre	Valeur typique
Taille d'entrée	46
Taille de sortie	81
Architecture cachée	[512, 81]
Taux d'apprentissage initial	0.3
Taille de batch	32
Nombre d'époques	200

TABLE 8.2 – Hyperparamètres du modèle

8.5 Sauvegarde et chargement des modèles

Le système implémente :

- **Sauvegarde automatique** : après chaque époque dans un fichier binaire
- **Format compact** : utilisation de `bincode` pour sérialiser la structure complète

- **Reprise d'entraînement** : possibilité de charger un modèle existant et continuer l'entraînement

Chapitre 9

Interface utilisateur et mode jeu

9.1 Commandes principales

Le programme offre deux modes d'utilisation :

9.1.1 Mode entraînement

```
cargo run --release cm <model_save_dir>
```

Ce mode :

- Charge le dataset depuis `dataset/fanorona/all.txt`
- Entraîne le modèle pour le nombre d'époques spécifié
- Sauvegarde automatiquement après chaque époque
- Affiche les métriques (perte, temps d'exécution)

9.1.2 Mode jeu

```
cargo run --release play <model_path>
```

Ce mode :

- Charge un modèle pré-entraîné
- Permet à l'utilisateur de jouer contre l'IA
- Affiche le plateau après chaque coup
- Valide les coups de l'utilisateur

9.2 Format des coups

Les coups sont spécifiés sous forme de deux entiers séparés par un espace :

- Premier entier : case de départ (0-8)
- Deuxième entier : case d'arrivée (0-8)

Les positions sont numérotées de gauche à droite, de haut en bas :

$$\begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}$$

Annexe A

Dérivations mathématiques complètes

A.1 Récapitulatif des notations

- $\mathbf{a}^{(0)} = \mathbf{x} \in \mathbb{R}^{n_0}$: vecteur d'entrée (avec $n_0 = 46$)
- Pour $k = 1, \dots, L$: couche k de dimension m_k
- Couche de sortie : $m_L = 81$
- $\mathbf{z}^{(k)} = W^{(k)}\mathbf{a}^{(k-1)} + \mathbf{b}^{(k)}$, $\mathbf{a}^{(k)} = \phi^{(k)}(\mathbf{z}^{(k)})$
- Étiquette : $\mathbf{y} \in \{0, 1\}^{81}$ avec une composante égale à 1 à l'indice $9d^* + a^*$
- $\boldsymbol{\delta}^{(k)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(k)}} \in \mathbb{R}^{m_k}$

A.2 Dérivées explicites pour la couche de sortie ($k = L$)

Gradient de sortie détaillé

Pour $i \in \{0, 1, \dots, 80\}$: $\delta_i^{(L)} = a_i^{(L)} - y_i$

Les gradients des paramètres de la couche de sortie sont :

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(L)}} = \delta_i^{(L)} \cdot a_j^{(L-1)}, \quad i \in \{0, \dots, 80\}, j \in \{0, \dots, m_{L-1} - 1\} \quad (\text{A.2.1})$$

$$\frac{\partial \mathcal{L}}{\partial b_i^{(L)}} = \delta_i^{(L)} \quad (\text{A.2.2})$$

A.3 Dérivées pour les couches cachées ($k < L$)

Rétropropagation complète avec ReLU

Pour $k \in \{L-1, L-2, \dots, 1\}$: $\delta_i^{(k)} = \left(\sum_{j=0}^{m_{k+1}-1} W_{ji}^{(k+1)} \delta_j^{(k+1)} \right) \cdot \mathbb{1}_{z_i^{(k)} > 0}$
 Les gradients des paramètres sont :

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(k)}} = \delta_i^{(k)} \cdot a_j^{(k-1)} \quad (\text{A.3.1})$$

$$\frac{\partial \mathcal{L}}{\partial b_i^{(k)}} = \delta_i^{(k)} \quad (\text{A.3.2})$$

A.4 Formulation matricielle par batch

Pour un batch de taille m :

$$\Delta^{(L)} = A^{(L)} - Y \in \mathbb{R}^{m \times 81} \quad (\text{A.4.1})$$

$$\Delta^{(k)} = (\Delta^{(k+1)} (W^{(k+1)})^T) \odot \mathbb{1}_{Z^{(k)} > 0}, \quad k = L-1, \dots, 1 \quad (\text{A.4.2})$$

$$\nabla_{W^{(k)}} \mathcal{L} = \frac{1}{m} (A^{(k-1)})^T \Delta^{(k)} \quad (\text{A.4.3})$$

$$\nabla_{\mathbf{b}^{(k)}} \mathcal{L} = \frac{1}{m} \sum_{i=1}^m \Delta_{i,:}^{(k)} \quad (\text{A.4.4})$$

où $\odot$ désigne le produit de Hadamard (élément par élément).

Annexe B

Extensions et perspectives

B.1 Améliorations possibles

B.1.1 Techniques d'optimisation avancées

- **Adam optimizer** : adaptation automatique du taux d'apprentissage avec moments
- **Learning rate scheduling** : décroissance progressive ou cyclique
- **Batch normalization** : normalisation des activations entre couches
- **Dropout** : régularisation par masquage aléatoire de neurones
- **Gradient clipping** : limitation de la norme du gradient

B.1.2 Architectures alternatives

- **Réseaux convolutifs** : exploitation de la structure spatiale du plateau
- **Réseaux résiduels** : connexions directes pour faciliter l'entraînement profond
- **Attention mechanisms** : focus adaptatif sur les régions importantes du plateau
- **Architectures plus profondes** : augmentation du nombre de couches cachées

B.1.3 Amélioration de la représentation

- **Features additionnelles** : mobilité, contrôle du centre, menaces
- **Encodage positionnel** : coordonnées spatiales explicites
- **Représentation symétrique** : augmentation des données par symétrie

B.2 Applications à d'autres jeux

L'architecture développée est générique et peut être adaptée à :

- **Fanorona** sur plateaux plus grands (5×9 traditionnel)
- **Autres jeux de plateau** : échecs simplifiés, dames, etc.

- **Jeux combinatoires** : avec encodage approprié de l'état

B.3 Apprentissage par renforcement

Une extension naturelle serait l'utilisation de techniques d'apprentissage par renforcement :

- **Self-play** : le modèle joue contre lui-même pour s'améliorer
- **Q-learning** : apprentissage de la fonction de valeur
- **Policy gradient** : optimisation directe de la politique
- **AlphaZero-like** : combinaison MCTS + réseau neuronal

Annexe C

Conclusion

Ce document a présenté une analyse exhaustive de l'architecture et de l'implémentation d'un réseau de neurones pour le jeu Fanorona 3×3 . L'approche développée combine une fondation mathématique rigoureuse avec une implémentation efficace en Rust, démontrant l'applicabilité des techniques d'apprentissage profond aux jeux traditionnels.

Les contributions principales incluent :

1. Une formulation mathématique complète des algorithmes de propagation avant et arrière
2. Une implémentation matricielle par batch pour des performances optimales
3. L'utilisation de l'activation ReLU dans les couches cachées pour éviter le gradient vanishing
4. Une architecture modulaire et extensible en Rust
5. Une analyse détaillée de la stabilité numérique et des choix d'implémentation
6. Des perspectives d'extension vers des problèmes plus complexes

Résultats clés :

- Architecture flexible paramétrable par taille de couches
- Encodage one-hot préservant la structure catégorielle
- Optimisation par mini-batch avec opérations matricielles
- Système de sauvegarde/chargement pour la reprise d'entraînement
- Interface de jeu interactive pour validation qualitative

L'architecture générique proposée offre une base solide pour l'exploration de variantes plus sophistiquées et l'application à d'autres jeux de stratégie combinatoire. Les fondements mathématiques détaillés permettent une compréhension approfondie des mécanismes d'apprentissage et facilitent les extensions futures.

Perspectives futures :

L'intégration de techniques d'apprentissage par renforcement, l'exploration d'architectures plus profondes, et l'application à des plateaux de taille standard (5×9) constituent des directions prometteuses pour poursuivre ce travail.